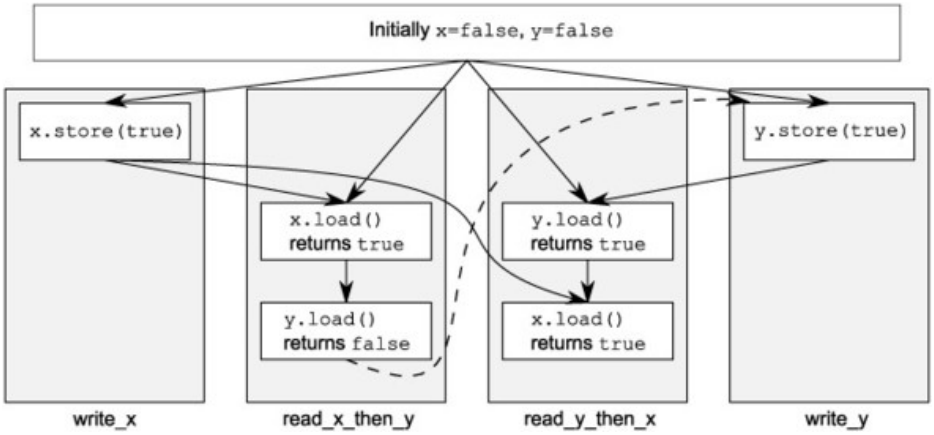
this point. Because the semantics of `memory_order_seq_cst` require a single total ordering over all operations tagged `memory_order_seq_cst`, there's an implied ordering relationship between a load of `y` that returns `false` ❸ and the store to `y` ❶. For there to be a single total order, if one thread sees `x==true` and then subsequently sees `y==false`, this implies that the store to `x` occurs before the store to `y` in this total order.

Of course, because everything is symmetrical, it could also happen the other way around, with the load of `x` ❹ returning `false`, forcing the load of `y` ❸ to return `true`. In both cases, `z` is equal to 1. Both loads can return `true`, leading to `z` being 2, but under no circumstances can `z` be zero.

The operations and happens-before relationships for the case that `read_x_then_y` sees `x` as `true` and `y` as `false` are shown in figure 5.3. The dashed line from the load of `y` in `read_x_then_y` to the store to `y` in `write_y` shows the implied ordering relationship required in order to maintain sequential consistency: the load must occur before the store in the global order of `memory_order_seq_cst` operations in order to achieve the outcomes given here.

Figure 5.3. Sequential consistency and happens-before



Sequential consistency is the most straightforward and intuitive ordering, but it's also the most expensive memory ordering because it requires global synchronization between all threads. On a multiprocessor system this may require quite extensive and time-consuming communication between processors.

In order to avoid this synchronization cost, you need to step outside the world of sequential consistency and consider using other memory orderings.

Non-Sequentially Consistent Memory Orderings

Once you step outside the nice sequentially consistent world, things start to get complicated. Probably the single biggest issue to come to grips with is the fact that *there's no longer a single global order of events*. This means that different threads can see different views of the same operations, and any mental model you have of operations from different threads neatly